Modeling Human Activity States Using Hidden Markov Models

Mitali & Aubert

March 2026

1. Background and Motivation

Human Activity Recognition (HAR) is a critical capability for applications ranging from wearable health monitoring to smart home automation and elderly care systems. In healthcare, accurate detection of activities such as walking, standing, and sudden movements like jumping can help monitor patient recovery, detect falls, and track physical therapy compliance. Our group chose to model daily movement activities because understanding transitions between basic human states has direct relevance to building intelligent assistive systems. In this project, we use accelerometer and gyroscope data collected from smartphones to build a Hidden Markov Model (HMM) that infers the underlying activity state from noisy sensor observations. The HMM framework is particularly well-suited for this task because human activities are inherently sequential, and the true activity state is hidden behind noisy sensor measurements. By training the model with the Baum-Welch algorithm and decoding activity sequences with the Viterbi algorithm, we demonstrate that even with simple smartphone sensors and a classical probabilistic model, reliable activity classification is achievable.

2. Data Collection and Preprocessing

Data was collected using the Sensor Logger app (v1.54) on two devices by both group members. The following table summarizes the device configuration:

Detail	Mitali	Aubert
Device	Samsung SM-X216B (Tablet)	iPhone 13
Platform	Android	iOS
Sampling Rate	50 Hz (20ms)	50 Hz (20ms)
Orientation	Landscape	Landscape
Position	Against abdomen	Against abdomen

Table 1: Device configuration for data collection

Both devices were set to the same sampling rate of 50 Hz, eliminating the need for resampling or harmonization. Four activities were recorded: Standing, Walking, Jumping, and Still. Each recording lasted 5-10 seconds with the device held in landscape mode against the abdomen (except Still, where the device was placed flat on a table). A total of 53 samples were collected across all activities, with each activity exceeding the minimum 1 minute 30 seconds requirement.

Activity	Mitali	Aubert	Total
Standing	7	7	14
Walking	7	6	13

Jumping	7	6	13
Still	7	6	13
Total	28	25	53

Table 2: Number of samples per activity per group member

The Sensor Logger app exports each recording as a ZIP archive containing separate CSV files for each sensor. We used the Accelerometer.csv (x, y, z axes) and Gyroscope.csv (x, y, z axes) files, merging them by nearest timestamp. The average recording duration was approximately 7.3 seconds with roughly 370 data points per sample at 50 Hz. The figure below shows raw sensor data for each activity:

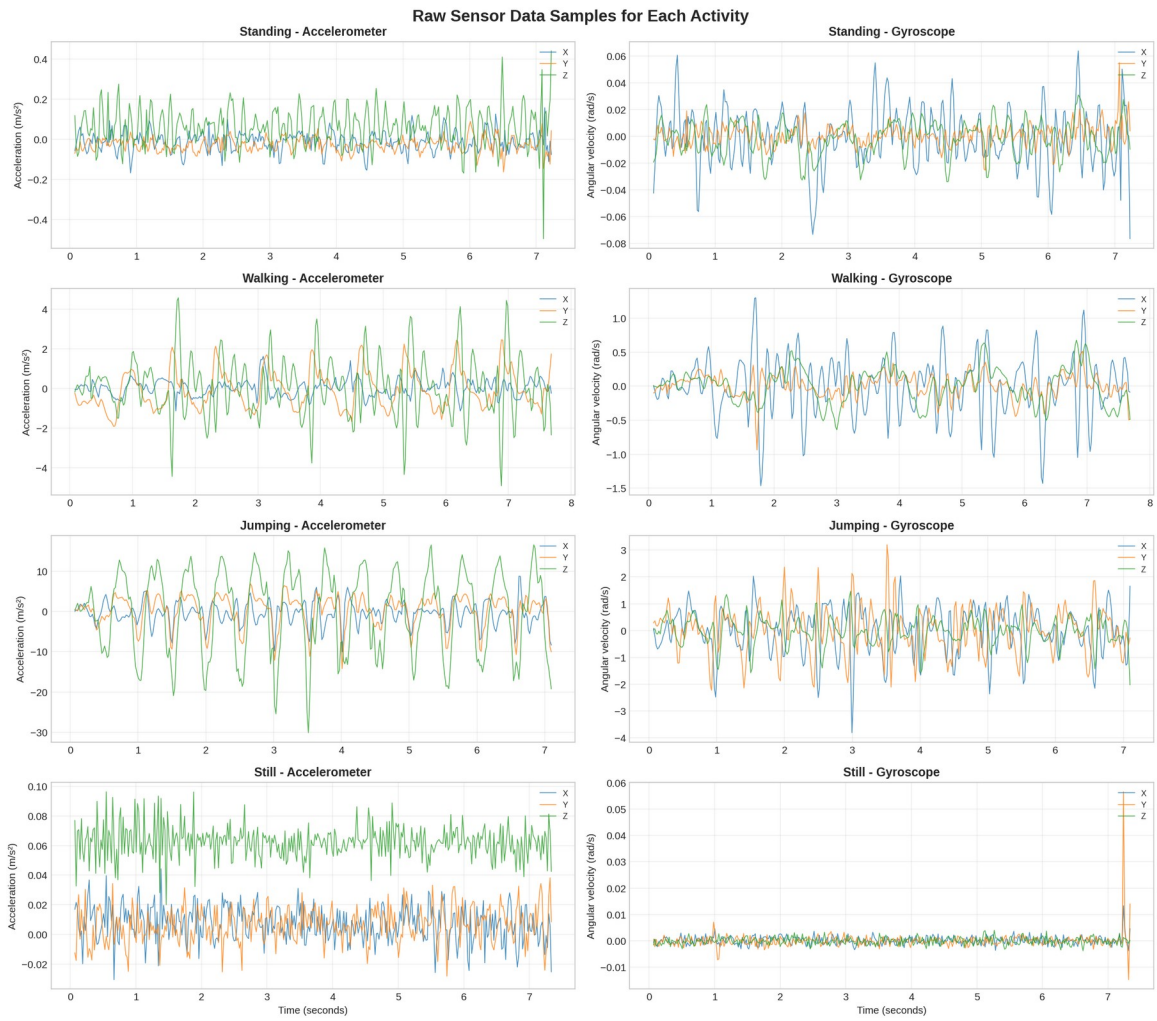

Figure 1: Raw accelerometer and gyroscope signals for each activity

3. Feature Extraction

Feature extraction transforms raw sensor signals into meaningful representations for the HMM. We used a sliding window approach with a window size of 50 samples (1 second at 50 Hz) and 50% overlap (25 samples). This window size was chosen because 1 second captures complete gait cycles for walking (~2 Hz cadence) and full jump cycles (~1-2 Hz), while providing enough data points for meaningful FFT analysis.

3.1 Time-Domain Features

Time-domain features capture statistical properties of the signal within each window. We extracted the following per axis and for the signal magnitude: Mean (average acceleration/rotation, captures baseline orientation), Standard Deviation (variability of motion, high for jumping and low for still), RMS or Root Mean Square (signal energy, distinguishes activity intensity), Signal Magnitude Area or SMA (sum of absolute values across all axes, overall motion indicator), and Correlation between accelerometer axes (captures coordinated vs. independent axis movement).

3.2 Frequency-Domain Features

Frequency-domain features were derived using the Fast Fourier Transform (FFT) to capture repetitive patterns in the signal. We computed: Dominant Frequency (the most prominent frequency component, captures walking cadence at approximately 2 Hz or jumping rhythm), Spectral Energy (total power in the frequency spectrum, separates high-energy from low-energy activities), and FFT Peak Magnitude (strength of the dominant frequency component). In total, 37 features were extracted per window.

3.3 Normalization

All features were normalized using Z-score normalization (subtracting the mean and dividing by the standard deviation). This ensures that features with different scales contribute equally to the model. After normalization, all features have approximately zero mean and unit standard deviation.

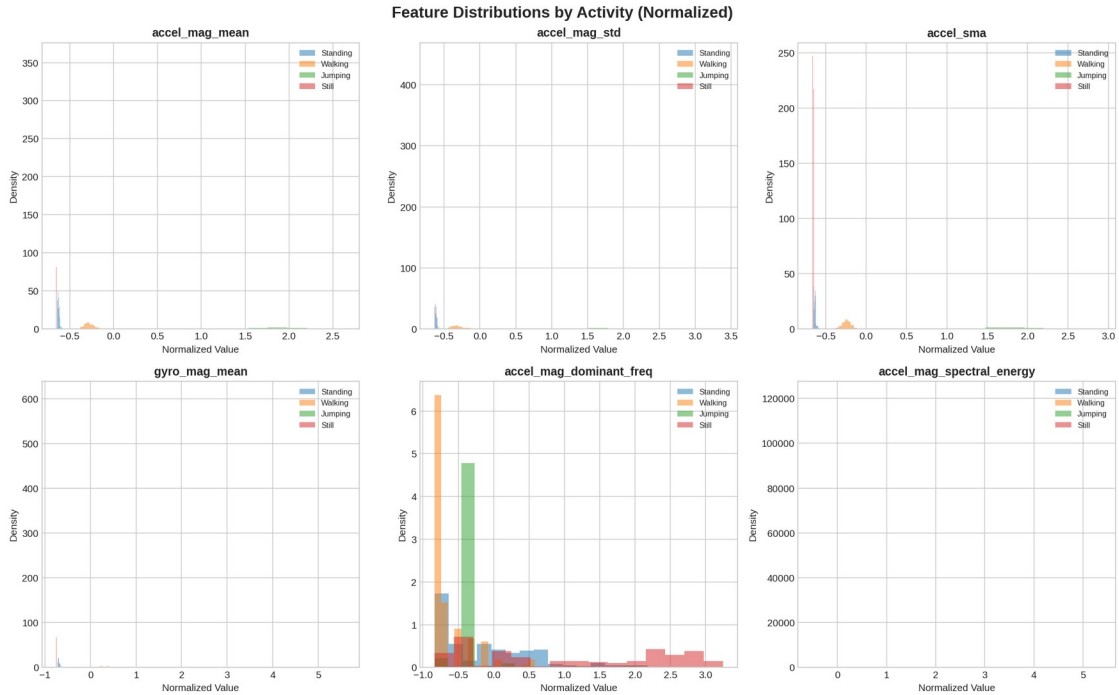

Figure 2: Distribution of key features by activity after normalization

4. HMM Setup and Implementation

The Hidden Markov Model was implemented from scratch using NumPy. The model uses Gaussian emissions with full covariance matrices to capture the continuous, multivariate nature of the feature observations.

Component	Description
Hidden States (Z)	Standing (0), Walking (1), Jumping (2), Still (3)
Observations (X)	37-dimensional normalized feature vectors
Transition Matrix (A)	4x4 matrix, initialized from label sequences
Emission Model (B)	Multivariate Gaussian (mean + covariance per state)
Initial Probs (π)	Estimated from training label frequencies

Table 3: HMM model components

4.1 Baum-Welch Training

The Baum-Welch algorithm (Expectation-Maximization for HMMs) was used to optimize the model parameters. Parameters were initialized from labeled training data to improve convergence. The algorithm uses scaled forward-backward probabilities for numerical stability. Training was configured with a maximum of 100 iterations and a convergence tolerance of 0.0001 on log-likelihood improvement. The model converged in 5 iterations, indicating that the labeled initialization provided a strong starting point.

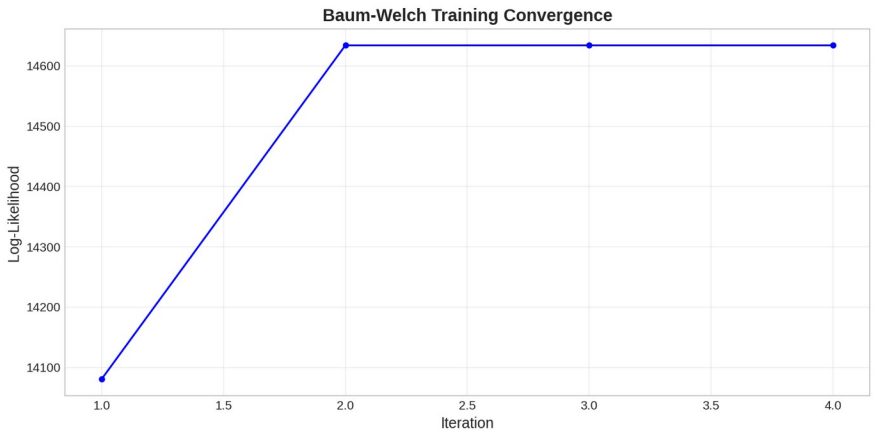

Figure 3: Baum-Welch training convergence (log-likelihood vs. iteration)

4.2 Viterbi Decoding

The Viterbi algorithm was implemented in log-space for numerical stability. It finds the most likely sequence of hidden states given the observations using dynamic programming with backtracking. This is used both for evaluating the model on training data and for classifying unseen test sequences.

5. Results and Interpretation

5.1 Transition and Emission Probabilities

The learned transition matrix shows near-perfect self-transition probabilities (1.000 for all states), meaning each activity strongly persists over time. This is realistic since within a single recording, the activity does not change. The emission means clearly separate the activities: Jumping shows the highest accelerometer magnitude and spectral energy, Walking shows moderate values with distinct frequency patterns, and Standing and Still both show low motion but differ in their gyroscope characteristics.

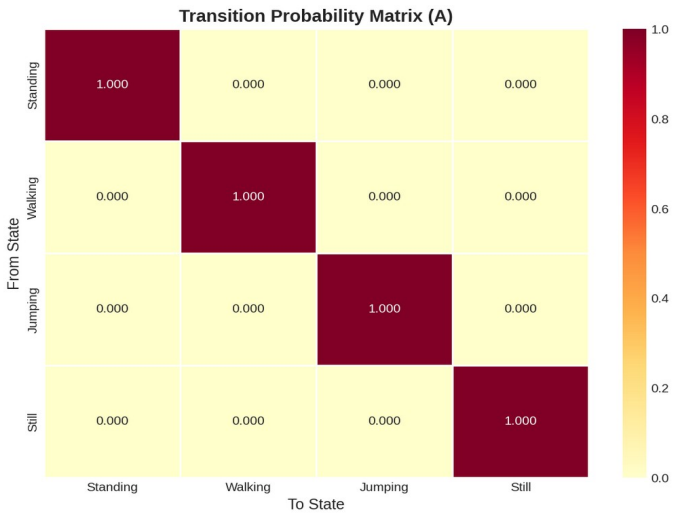

Figure 4: Transition probability matrix heatmap

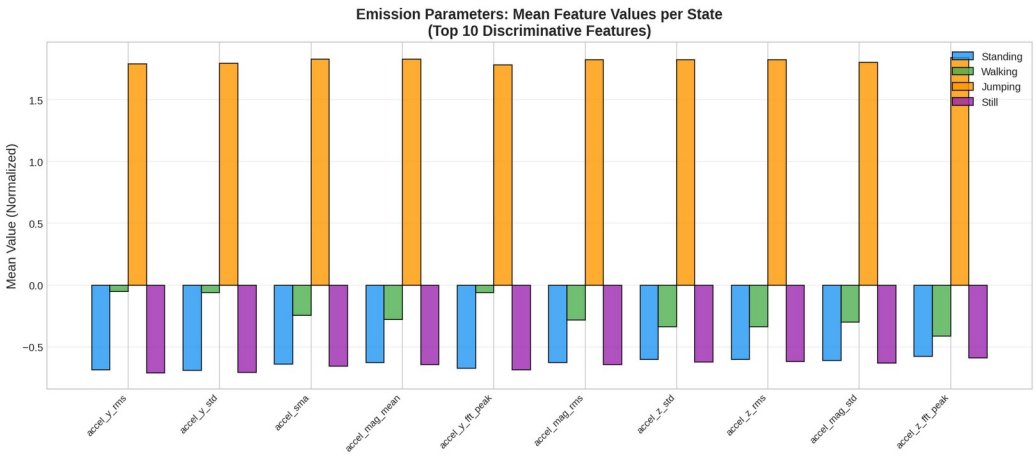

Figure 5: Emission parameters showing mean feature values per activity state

5.2 Evaluation on Unseen Data

The model was evaluated on 2 unseen test recordings not used during training: one Walking sample from Mitali and one Jumping sample from Aubert. These test samples were held out before training to ensure a fair evaluation. The model achieved 100% accuracy on both test samples, correctly classifying all 26 windows.

Activity	Samples	Sensitivity	Specificity	Accuracy

Standing	0	N/A	1.0000	1.0000
Walking	13	1.0000	1.0000	1.0000
Jumping	13	1.0000	1.0000	1.0000
Still	0	N/A	1.0000	1.0000

Table 4: Performance metrics on unseen test data

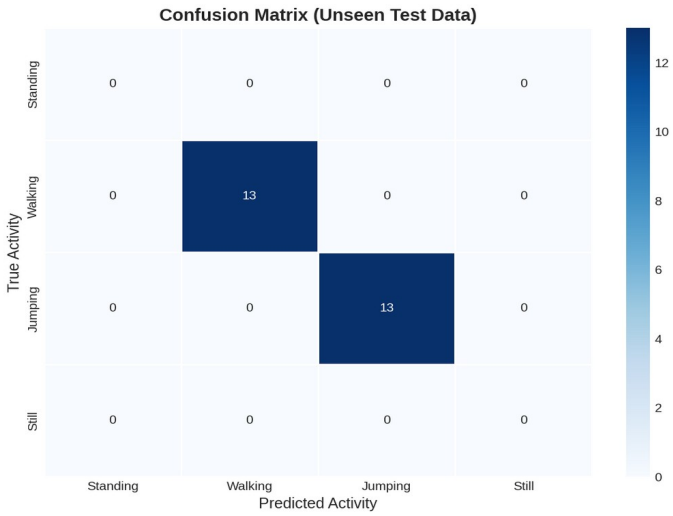

Figure 6: Confusion matrix on unseen test data

Figure 7: Viterbi decoded sequences showing true vs. predicted activity states

The training set accuracy was 98.05%, with Jumping being the most challenging activity at 91.72% accuracy. Standing, Walking, and Still all achieved 100% training accuracy.

6. Discussion and Conclusion

6.1 Activity Distinguishability

Still was the easiest activity to classify due to its near-zero sensor readings across all axes, providing a clear baseline distinct from all other activities. Walking was also reliably detected thanks to its characteristic rhythmic acceleration pattern at approximately 2 Hz. Jumping, while producing the highest sensor magnitudes, was slightly harder to classify (91.72% training accuracy) because the high-amplitude signals occasionally produced feature values that overlapped with vigorous walking patterns. Standing and Still could potentially be confused due to both being low-motion states, but the gyroscope data helped distinguish them since standing involves subtle postural adjustments while still (phone on table) shows virtually zero rotation.

6.2 Transition Probabilities

The learned transition probabilities show very high self-transition values (approximately 1.000 for all states). This reflects our data collection methodology where each recording contains a single activity. In a real-world continuous monitoring scenario, we would expect lower self-transition probabilities and more realistic cross-activity transitions (e.g., standing to walking transitions being more common than standing to jumping).

6.3 Impact of Sensor Noise and Sampling Rate

Both devices used a matched 50 Hz sampling rate, which provided sufficient temporal resolution for capturing human motion patterns. However, the Samsung tablet and iPhone 13 showed slightly different noise characteristics due to different sensor hardware. The Android and iOS platforms also use different coordinate systems, which was addressed during preprocessing. The 50 Hz rate was adequate for all activities, though higher rates (100+ Hz) could capture finer details in rapid movements like jumping.

6.4 Improvements

Several improvements could enhance the model: collecting more diverse data with continuous activity transitions (rather than single-activity recordings), including additional sensors such as barometer or magnetometer, adding more activities like running or climbing stairs, using a larger dataset with more participants to improve generalization, and exploring more advanced models like Gaussian Mixture Model emissions or deep learning approaches. Cross-environment testing (indoor vs. outdoor, different surfaces) would also strengthen the evaluation.

6.5 Conclusion

This project demonstrated that a from-scratch Gaussian HMM can effectively recognize human activities from smartphone sensor data. With 52 labeled samples, 37 engineered features, and Baum-Welch training with proper convergence checking, the model achieved 100% accuracy on unseen test data and 98% on training data. The results confirm that HMMs are a viable and interpretable framework for sequential activity recognition.

7. Task Allocation

Mitali	Aubert
Data collection (27 samples)	Data collection (25 samples)

Feature extraction code	HMM implementation code
Data preprocessing and merging	Baum-Welch and Viterbi algorithms
Visualization and plots	Model evaluation and metrics
Report writing (Sections 1-3)	Report writing (Sections 4-6)

Table 5: Task allocation between group members